Description Submission view

Available from: Cuma, 27 Şubat 2026, 1:30 PM
Due date: Cumartesi, 14 Mart 2026, 11:59 PM
Requested files: Assignment1.java (Download)
Type of work: Individual work

AIRPORT GATE MANAGER SYSTEM

Please read this manual carefully before beginning your implementation in the Moodle VPL environment. You are expected to build several classes that interact with each other to manage flights at an airport terminal.
System Architecture and Class Overview
Your system will model different types of flights and an airport to manage them. You will use an inheritance hierarchy where a general flight class shares its properties with more specific flight types.

1. Class: Flight (Base Class)
This is the parent class representing a generic flight.

- **Attributes:**
  - An integer representing the flight code.
  - An integer representing the minutes remaining until departure.
- **Constructor:**
  - Takes two parameters: int flightCode and int minutesToDeparture to initialize the attributes. *(Note: These values can initially be 0, or even logically negative if a flight is already considered departed).*
- **Methods:**
  - Implement standard getter and setter methods for both attributes.
    - getFlightCode() returning an int.
    - setFlightCode(int flightCode) returning void.
    - getMinutesToDeparture() returning an int.
    - setMinutesToDeparture(int minutesToDeparture) returning void.

2. Class: DomesticFlight (Child Class)
This class extends the Flight base class.

- **Attributes:**
  - An integer representing the terminal zone.

?

- **Constructor:**
  - Takes three parameters: int flightCode, int minutesToDeparture, and int terminalZone. You must use the super() keyword to initialize the inherited attributes.
- **Methods:**
  - getTerminalZone() returning an int.

3. Class: InternationalFlight (Child Class)
This class also extends the Flight base class.

- **Attributes:**
  - An integer representing the time required for customs clearance (customsTime). *(Note: This value can be exactly 0).*
- **Constructor:**
  - Takes three parameters: int flightCode, int minutesToDeparture, and int customsTime. Again, utilize the super() keyword for the parent attributes.
- **Methods:**
  - getActualDepartureTime() returning an int. This method calculates and returns the total time before the plane actually leaves.
  - **Crucial Rule:** Do not calculate and store this total time in the constructor! You must dynamically calculate it (by adding the inherited minutesToDeparture to the customsTime) *every time* this method is called. This ensures the total time remains accurate even if the inherited minutes are changed later via a setter method.

The Manager Class: Airport
This class is the core engine of your simulation. It manages two separate fixed-size arrays representing the physical gates at the airport: one for domestic flights and one for international flights.

- **Constructor:**
  - Takes two parameters: int domesticCapacity and int intlCapacity. It initializes the internal arrays using these capacities. *(Note: The system should not crash if an airport is built with a capacity of 0!).*
- **Methods:**
  - **assignGate(DomesticFlight df) returning boolean:** This is an overloaded method. It iterates through the domestic gates array from the beginning (index 0). It finds the first empty spot (a null index), assigns the provided flight object to it, and returns true. If the array is completely full, it returns false.
  - **assignGate(InternationalFlight itf) returning boolean:** This is the second version of the overloaded method. It performs the exact same logic as above, but operates exclusively on the international gates array.
  - **updateTime(int minutesPassed) returning void:**
    This simulates the passage of time. It iterates through *both* arrays. For every active flight, it subtracts the minutesPassed from the flight's current minutes to departure.

- **Departure Condition:** If a flight's remaining time drops to **exactly 0 or below**, it has departed! You must free up the gate by setting that array index back to null.
- **Safety Check:** Make sure your loops safely skip over already empty (null) gates without throwing a NullPointerException.

- **findNextAvailableDomesticGate() returning int:**
  This method helps air traffic control decide where to put the next domestic plane. It scans the domestic array and returns the index of the first empty gate (null). If the airport is completely full, it returns the index of the gate holding the flight scheduled to leave the *soonest* (the minimum minutes to departure).
  - **Tie-Breaker Rule:** If the airport is full and there are *multiple* flights sharing the exact same minimum departure time, your method must return the index of the **first** one it encounters in the array (the lowest index among the ties).
- **applyCascadingDelay(int delayAmount) returning int:**
  Due to bad weather, you need to delay flights. This is an iterative method that loops through the entire domestic gates array. It adds the delayAmount (in minutes) to every active (non-null) domestic flight it finds. It must return the total count of flights that were successfully delayed.

Example Simulation Scenario

To help you visualize the expected behavior, consider this scenario:

1. **Setup:** You create an Airport with a domestic capacity of 4.
2. **Assignment:** You assign three domestic flights. They take up indices 0, 1, and 2. Index 3 remains empty (null).
3. **Time Simulation:** You call updateTime(20). The system updates all active flights. The flight at index 1 drops to exactly 0 minutes left. It departs, and index 1 becomes null.
4. **Finding a Gate:** The domestic terminal now has flights at index 0 and 2. Indices 1 and 3 are empty. You call findNextAvailableDomesticGate(). The system checks the array and returns 1 because it is the *first* available empty gate encountered.
5. **Delay Simulation:** Bad weather hits! You call applyCascadingDelay(30) to add a 30-minute delay. The loop checks index 0 (delays it), checks index 1 (skips it because it's null), checks index 2 (delays it), and checks index 3 (skips it). It correctly returns that it successfully delayed 2 active flights.

Final Checklist & Edge Cases for VPL

To successfully pass all the hidden evaluation tests, double-check your code against these specific scenarios:

- [ ] Did you use super() correctly in child constructors?
- [ ] Does InternationalFlight dynamically calculate the actual departure time?
- [ ] Does updateTime() properly clear gates when the time is <= 0 without crashing on completely empty airports?
- [ ] Does findNextAvailableDomesticGate() correctly handle "ties" by picking the first matching flight index?
- [ ] Does your iterative applyCascadingDelay() method safely skip null gaps in the middle of the array, only counting active flights?

Example Output:

```
--- AIRPORT GATE MANAGER TESTING ---

--- Testing Child Methods ---
Actual Intl Departure Time (Expected 45): 45

--- Testing Gate Assignment ---
Assign D1 (Expected true): true
Assign D4 to full airport (Expected false): false
Assign I1 (Expected true): true

--- Testing Next Available Gate ---
Next available domestic gate index (Expected 0): 0

--- Testing Time Update & Auto-Drop ---
Simulating 20 minutes passing...
Next available domestic gate index (Expected 0): 0
Assign D4 now (Expected true): true

--- Testing Cascading Delay (Iteration) ---
Total domestic flights delayed (Expected 3): 3
D2's new departure time (Expected 55): 55
```

Good luck with your implementation!

Requested files

Assignment1.java

```java
public class Assignment1 {
    public static void main(String[] args) {
        System.out.println("--- AIRPORT GATE MANAGER TESTING ---");

        // 1. Setup
        Airport myAirport = new Airport(3, 2);

        DomesticFlight d1 = new DomesticFlight(101, 15, 1);
        DomesticFlight d2 = new DomesticFlight(102, 45, 2);
        DomesticFlight d3 = new DomesticFlight(103, 120, 1);
        DomesticFlight d4 = new DomesticFlight(104, 60, 3); // Extra flight for testing full capacity

        InternationalFlight i1 = new InternationalFlight(901, 30, 15);

        // 2. Test InternationalFlight Child Method
        System.out.println("\n--- Testing Child Methods ---");
        System.out.println("Actual Intl Departure Time (Expected 45): " + i1.getActualDepartureTime());

        // 3. Test Gate Assignment
        System.out.println("\n--- Testing Gate Assignment ---");
        System.out.println("Assign D1 (Expected true): " + myAirport.assignGate(d1));

        // Populate array to test capacity
        myAirport.assignGate(d2);
        myAirport.assignGate(d3);

        System.out.println("Assign D4 to full airport (Expected false): " + myAirport.assignGate(d4));
        System.out.println("Assign I1 (Expected true): " + myAirport.assignGate(i1));

        // 4. Test Best Gate Logic (Airport is full, D1 at index 0 leaves soonest in 15 mins)
        System.out.println("\n--- Testing Next Available Gate ---");
        System.out.println("Next available domestic gate index (Expected 0): " + myAirport.findNextAvailableDomesticGate());

        // 5. Test Update Time & Auto-Drop
        System.out.println("\n--- Testing Time Update & Auto-Drop ---");
        System.out.println("Simulating 20 minutes passing...");
        myAirport.updateTime(20);

        // D1 (15 mins) should have dropped. Gate 0 should be empty.
        System.out.println("Next available domestic gate index (Expected 0): " + myAirport.findNextAvailableDomesticGate());

        // Assign D4 again, it should now take the empty spot at index 0
        System.out.println("Assign D4 now (Expected true): " + myAirport.assignGate(d4));

        // 6. Test Iteration
        System.out.println("\n--- Testing Cascading Delay (Iteration) ---");
        // Currently, D4 (60 mins), D2 (45-20=25 mins), and D3 (120-20=100 mins) are at the gates
        int delayedCount = myAirport.applyCascadingDelay(30);
        System.out.println("Total domestic flights delayed (Expected 3): " + delayedCount);
        System.out.println("D2's new departure time (Expected 55): " + d2.getMinutesToDeparture());
    }
}

//CODE HERE
```

VPL